

oslab-project4

Design Review

地址

Name	Address
Segment0 BBL代码及内存	0xffffffffc0_50000000 – 0xffffffffc0_50200000
Segment1 Kernel 的数据段/代码段	0xffffffffc0_50200000 – 0xffffffffc0_51000000
Segment2 Kernel 页表以及跳转表等	0xffffffffc0_51000000 – 0xffffffffc0_52000000
Segment3 供内核动态分配使用的内核虚拟地址空间	0xffffffffc0_52000000 – 0xffffffffc0_60000000
bootloader 入口	0x50200000 ∈ PhysicalSeg1
PGDIR_PA	0x51000000 ∈ PhysicalSeg2
bootblock taskinfo 地址	0x50500000 ∈ PhysicalSeg1
bootblock argv 地址	0x50700000 ∈ PhysicalSeg1
hart#0 boot_kernel() 栈空间	0x50fff000 – 0x51000000 ∈ Seg1
hart#1 boot_kernel() 栈空间	0x50ffe000 – 0x50fff000 ∈ Seg3
hart#0 init/main() 栈空间	0xf*c052000000 – 0xf*c052001000 ∈ Seg3
hart#1 init/main() 栈空间	0xf*c052001000 – 0xf*c052002000 ∈ Seg3
init/main() 入口	0xf*c0_50202000 ∈ Seg1
kernel jumptab	0xf*c0_51ffff00 ∈ Seg2

内核启动流程

bootblock::main → (start.S) _boot → (boot.c) boot_kernel → (head.S) _start → init::main

- [build/bootblock, 0x50200000] bootblock::main: hart=0则开始搬运内核, 加载taskinfo, 然后跳转至 _boot, 否则 wait_for_wakeup
- [build/main, 0x50202000] _boot: 对照如上表格设置 per-hart boot stack, 把mhartid 放入 a0 后调用 boot_kernel
- [build/main] boot_kernel: 设置vm, 映射 Segment1 区域到相同的虚拟地址, 然后调用 _start

从此开始, 使用虚拟地址

- _start: 对照如上表格设置per-hart kernel stack, 若 hart=0, 清零bss, 对于所有hart都进入main
- init::main: 进入内核init进程, 双核都启动完后取消 Segment1 的临时映射

回顾链接过程

上面的这些地址是通过 Makefile 中的 `--defsym=TEXT_START=xxx` 传入链接器, 而链接脚本中会设置从 `TEXT_START` 开始

```
OUTPUT_FORMAT("elf64-littleriscv", "elf64-littleriscv",
               "elf64-littleriscv")
OUTPUT_ARCH("riscv")
ENTRY(_start)
SECTIONS
```

```

{
    . = TEXT_START;
    /* UCAS boot stage code */
    .text
    :
    {
        *(.bootkernel_entry)
        *(.bootkernel)
    }
    .text
    :
    {
        _ftext = . ;
        *(.entry_function)
        *(.text.unlikely .text.*_unlikely .text.unlikely.*)
        *(.text.exit .text.exit.*)
        *(.text.startup .text.startup.*)
        *(.text.hot .text.hot.*)
        *(.text .stub .text.* .gnu.linkonce.t.*)
        /* .gnu.warning sections are handled specially by elf32.em. */
        *(.gnu.warning)
    }
}

```

设置默认 entry 是 `_start` (可以被编译时 `-e entry` 覆盖, 现在 bootblock 的 entry 是 `main`, main 的 entry 是 `_boot`)

这个设置的 entry 只会影响 elf 头, 不会影响实际 elf 文件布局, 也就是说不会因为把 `entry` 设置成了 `_start` 就会让 `_start` 排在 `TEXT_START` 的位置

这是 `-e _start` 的编译效果:

```

$ readelf -h build/main
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                   EXEC (Executable file)
  Machine:                                RISC-V
  Version:                                0x1
  Entry point address:                    0xffffffffc050202190
  Start of program headers:                64 (bytes into file)
  Start of section headers:                304320 (bytes into file)
  Flags:                                   0x5, RVC, double-precision ABI
  Size of this header:                     64 (bytes)
  Size of program headers:                 56 (bytes)
  Number of program headers:                4
  Size of section headers:                 64 (bytes)
  Number of section headers:                24
  Section header string table index:       23

```

```

$ nm build/main | egrep "(_start|_boot)"
ffffffc05020e49c S __bss_start
ffffffc050202000 T _boot
ffffffc050202190 T _start
ffffffc05020e480 G scroll_start

```

可以看到 `_boot` 位置是正确的，只是 `Entry point address` 有点问题

实际的布局是按照 `riscv.lds` 中的顺序手动排列的

顺序如下：

```
.bootkernel_entry # _boot
.bootkernel # bootkernel相关函数
.entry_function # _start
```

数据结构设计：

`kva_t`: (kernel virtual address) 当前页表中的地址，可以直接访问
`uva_t`: (user virtual address) foreign 地址，需要结合一个 `pgdir` 来翻译，可能被换出到swap，为了不在 kernel 里面 `pagefault`，每次访问要检查一下
`uva_object_t`: 封装访问，自动检查

```
uva_object_t addr;
char x = addr.at<char>(index); // safe!
```

这里转换一遍 `uva2kva` 是为了不再kernel中直接访问user memory，因为被scheduler调度之后 `sstatus` 的 `SUM` 位可能被清掉了（恢复的是 用户态的 `sstatus`）

Bugs

1. 设置完 `satp` 直接炸了
A: `sp` 不在映射物理地址范围内，将 `boot_kernel` 的 `sp` 从 `0x52000000` 改成 `0x50000000~0x51000000` 之间
2. `enter kernel` 时有问题
考虑这个语法：

```
typedef void (*kernel_entry_t)(unsigned long);
extern uintptr_t _start;
{
    int mhartid = ...;
    (kernel_entry_t)(_start)(mhartid);
}
```

`_start` 只是一个地址是 `0x5xxxxxx` 的符号，它的内容是一堆指令，所以这里应该取它的地址，直接将 `_start` 设成指针类型也行
一个问题，这里似乎不会做函数指针衰减？

```
extern kernel_entry_t _start;
```

直接这么写，不取地址有点问题

3. 传参问题
现在 `bootblock` 和 `_start` 之间隔了个 `_boot`，怎么传递参数呢
A: 让编译器处理，直接写 `_start(argc, argv)`，只要让手写的汇编部分满足abi就行
4. 内核触发`pagefault`问题：现在是在`syscall`中直接打开`user_memory`，但是会存在一个问题：如果`syscall`中访问了不存在的memory，触发`page fault`，会再次`lock_kernel()`，导致死锁

A1: 放通 page fault, 不再 lock kernel, 但这样需要仔细考虑所有用到page的地方, 并且page更新的时候也可能触发sched (页框数存不下页目录, 杀掉相关进程), 如果要这么写得整个重构成细粒度锁, 不可行
A2:

- 其实有一个 workaround: 在 syscall 的地方预先测试传入的 user memory 有没有问题, 如果没有分配, 那么unlock_kernel(), 然后访问一下触发page fault, 分配完了继续执行
- 可以直接手动操作页表分配, 不用unlock_kernel了

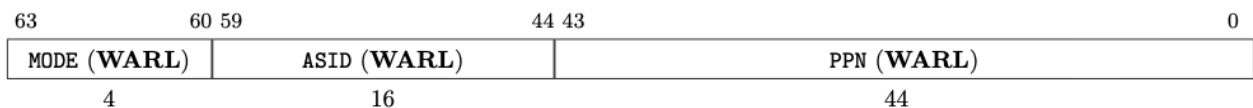
A3:

- A2的解决方法有问题, 一个是传入的 user va 区间可能大得离谱根本放不下物理内存, 二个是执行过程中可能被调度, 一旦被调度走了可能页就失效了。综上: 需要在每次访问 user memory 的时候都 check 一下
 - 对于所有的 `uva_t`, 其实kernel直接访问都是不安全的, 这个地址是在user空间, 可能被swap掉了, 不能直接访问, 每次访问都需要移进当前内存, 写一个辅助类 `uva_object_t` 自动完成 swapin 的过程

5. shell卡住问题: 由于do_mbox_recv执行地太久了, 而现在是一次syscall锁住整个内核, 会导致shell卡在某个核等待sys_getchar返回, 而sys_getchar等内核锁, 内核锁被send/recv持有, send/recv之间来回交换执行权, 但因为两个syscall都没结束, 不会释放内核锁, 导致getchar卡住

Appendix

csr.satp:



虚地址:

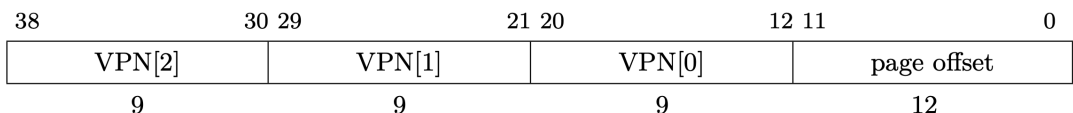


图 P4-3: Sv39 虚地址

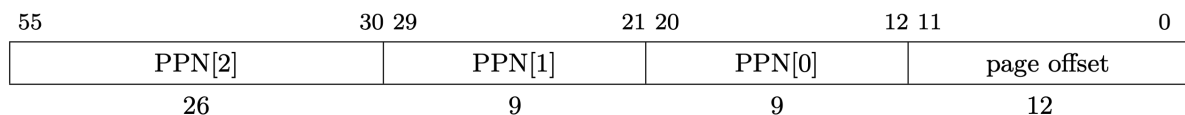


图 P4-4: Sv39 物理地址

sv39页表项

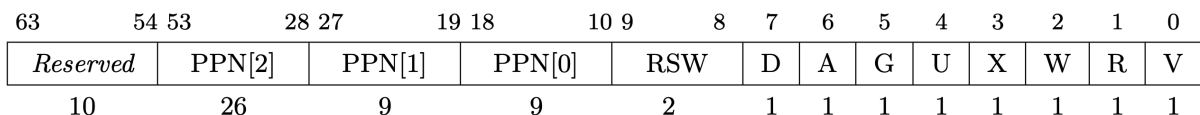


图 P4-5: Sv39 页表项

页大小 4K, 每个页表项 8B, 共 2⁹ 项, 虚地址9位一段正好塞满一页

